

Project 1 Report: Used Vehicle Price Prediction

Stacy Bowler
Utah State University
stacy.bowler@sdl.usu.edu

Derrik Miller
Utah State University
derrik.miller@sdl.usu.edu

2026-06-19

Abstract This project develops a multiple linear regression model to estimate used vehicle selling prices from vehicle characteristics, condition, mileage, model, and geographic location. The resulting model explains approximately 82% of the variation in observed selling prices and provides a framework for supporting vehicle acquisition, pricing, and inventory decisions.

Introduction

The objective of this project is to develop a predictive model capable of estimating the market value of used vehicles. Accurate vehicle valuation supports pricing decisions for dealerships, fleet operators, auction participants, and vehicle purchasing organizations.

The business decision addressed by this analysis is:

Given a used vehicle's characteristics, what selling price should be expected in the current market?

The resulting model can be used to identify vehicles that appear underpriced or overpriced relative to market expectations.

Business Objective

The primary objective is to minimize pricing error between predicted and observed vehicle sale prices.

A practical loss function for this problem is Mean Squared Error (MSE):

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

where:

- y_i is the observed selling price

- $\hat{y}_i$ is the predicted selling price

Lower prediction error improves decision quality by reducing the likelihood of overpaying for inventory or undervaluing vehicles during resale.

Plan

The analysis follows the regression modeling workflow:

1. Define the business problem.
2. Acquire and clean vehicle sales data.
3. Explore relationships between candidate predictors and selling price.
4. Build an interpretable regression model.
5. Evaluate assumptions and model fit.
6. Generate predictions and business recommendations.

The ideal dataset would contain:

- Vehicle age
- Odometer reading
- Mechanical condition
- Original MSRP
- Vehicle trim level
- Accident history
- Maintenance history
- Regional market conditions
- Vehicle Make/Model
- Sell price

Several of these variables were unavailable. Vehicle model and state were therefore used as proxy variables for differences in vehicle desirability and regional pricing effects. Vehicle make, age, and sell price should also act as a proxy for original MSRP.

The sales data was limited to 34 states. However, some states like Alaska had few observations.

Build

Data Source

The dataset was obtained from Kaggle at <https://www.kaggle.com/code/bavlyyoussef/vehicle-sales/input> and contains used vehicle transaction information. It's unclear how this data was collected or where it came from exactly.

Variables initially available included:

- Year
- Body
- Color
- Make
- Model

- MMR
- VIN
- Seller
- Transmission
- State
- Condition
- Odometer
- Selling price

Data Cleaning

The following preprocessing steps were performed:

- Removed records with missing values in critical fields.
- Calculated vehicle age from model year.
- Converted odometer values into units of 10,000 miles.
- Standardized categorical values.
- Grouped low-frequency vehicle models into manufacturer-specific “Other” categories.
- Grouped low-frequency states into broader state categories.
- Created dummy variables for categorical predictors.

```
import pandas as pd
import polars as pl
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np
import statsmodels.api as sm
import os
import pickle
from statsmodels.stats.outliers_influence import variance_inflation_factor
from sklearn.model_selection import train_test_split

raw_data = pl.read_csv('./data/kaggle_car_prices.csv')

make_map = {
    "vw": "volkswagen",
    "mercedes": "mercedes-benz",
    "mercedes-b": "mercedes-benz",
    "landrover": "land rover",
    "ford truck": "ford",
    "ford tk": "ford",
    "chev truck": "chevrolet",
    "gmc truck": "gmc",
    "mazda tk": "mazda",
    "dodge tk": "dodge",
```

```

    "hyundai tk": "hyundai",
}

body_map = {
    "regular-cab": "regular cab",

    "supercrew": "crew cab",
    "supercab": "extended cab",
    "crewmax cab": "crew cab",
    "double cab": "crew cab",

    "access cab": "extended cab",
    "club cab": "extended cab",
    "king cab": "extended cab",
    "xtracab": "extended cab",

    "cab plus": "extended cab",
    "cab plus 4": "extended cab",

    "e-series van": "van",
    "promaster cargo van": "van",
    "transit van": "van",
    "ram van": "van",

    "g sedan": "sedan",
    "g coupe": "coupe",
    "g convertible": "convertible",

    "genesis coupe": "coupe",
    "elantra coupe": "coupe",
    "q60 coupe": "coupe",
    "g37 coupe": "coupe",
    "cts coupe": "coupe",
    "cts-v coupe": "coupe",

    "beetle convertible": "convertible",
    "q60 convertible": "convertible",
    "g37 convertible": "convertible",
    "granturismo convertible": "convertible",
    "granturismo convertible": "convertible",

    "cts wagon": "wagon",
    "cts-v wagon": "wagon",

```

```

    "tsx sport wagon": "wagon",
}

pl.Config.set_tbl_rows(-1)
pl.Config.set_tbl_cols(-1)
pl.Config.set_fmt_str_lengths(100)

# Drop missing values in key columns
df = raw_data.drop_nulls(subset=['make', 'body', 'color', 'condition', 'year',
'saledate', 'sellingprice', 'state', 'transmission'])

# make, body, transmission, state, color need to be consistent and valid

# Clean 'make' column
df = df.with_columns(
    pl.col("make")
    .str.strip_chars()
    .str.to_lowercase()
)

df = df.with_columns(
    pl.col("make")
    .replace(make_map)
)

# Clean up body
df = df.with_columns(
    pl.col("body")
    .str.strip_chars()
    .str.to_lowercase()
)

df = df.with_columns(
    pl.col("body").replace(body_map)
)

# Clean up transmission, convert any not automatic/manual to null and then
drop them
df = df.with_columns(
    pl.when(
        ~pl.col("transmission")
        .str.to_lowercase()
        .is_in(["automatic", "manual"])
    )

```

```

    )
    .then(None)
    .otherwise(pl.col("transmission").str.to_lowercase())
    .alias("transmission")
)

df = df.drop_nulls(subset=["transmission"])

# Clean up color
df = df.with_columns(
    pl.when(pl.col("color") == "-")
        .then(pl.lit("unknown"))
        .otherwise(pl.col("color"))
        .alias("color")
)

# Extract 'sale_year' from 'saledate'
df = df.with_columns(
    sale_year=pl.col('saledate').str.extract(r'(\d{4})').cast(pl.Int64,
strict=False)
)

# Calculate vehicle age: sale_year - year
df = df.with_columns(
    vehicle_age=pl.col('sale_year') - pl.col('year')
)

# Filter out invalid ages and extreme odometers
df = df.filter(pl.col('vehicle_age') >= 0)
df = df.filter(pl.col('odometer') < 250000)
df = df.with_columns(
    (pl.col("odometer") / 10000).alias("odometer_10k")
)

y = df["sellingprice"]

features = [
    "vehicle_age",
    "odometer_10k",
    "condition",
    # "mmr", # We need to remove mmr as it's already an estimate for the price
of the vehicle
    # "make", # make is captured in model since a model like z3 is only for BMW

```

```

    "model",
    "state_grouped", # Added state to predict price variations by location
    # "body", # Body tends to be captured by the model as well
    # "transmission", # insignificant
    # "color", # color is not statistically significant and mostly just added noise
]

# Convert to pandas
pdf = df.to_pandas()

# Take a stratified sample of 5000 rows
sample_pdf, _ = train_test_split(
    pdf,
    train_size=5000,
    random_state=42,
    stratify=pdf["state"]
)

# Split sampled data into train/test 80/20% using a standard random sample
train_pdf, test_pdf = train_test_split(
    sample_pdf,
    test_size=0.2,
    random_state=42
)

# Tricky, some models are very rare and add a lot of extra variables to look at.
# With so few items it may start to memorize the data as well,
# so we will combine rare models together, while preserving the make still so
# like BMW_Other.
threshold = 20

model_counts = (
    train_pdf.groupby(['make', 'model'])
    .size()
    .reset_index(name='count')
)

common_models = set(
    zip(
        model_counts.loc[model_counts['count'] >= threshold, 'make'],
        model_counts.loc[model_counts['count'] >= threshold, 'model']
    )
)

```

```

def collapse_model(row):
    if (row['make'], row['model']) in common_models:
        return row['model']
    return f"{row['make']}_Other"

train_pdf['model'] = train_pdf.apply(collapse_model, axis=1)
test_pdf['model'] = test_pdf.apply(collapse_model, axis=1)

# Group states that don't have very many observations as well.
state_threshold = 30

state_counts = train_pdf["state"].value_counts()
kept_states = state_counts[state_counts >= state_threshold].index

def group_state(x):
    return x if x in kept_states else "Other"

train_pdf["state_grouped"] = train_pdf["state"].apply(group_state)
test_pdf["state_grouped"] = test_pdf["state"].apply(group_state)

X_train = pd.get_dummies(
    train_pdf[features],
    drop_first=True
)

X_test = pd.get_dummies(
    test_pdf[features],
    drop_first=True
)

X_test = X_test.reindex(
    columns=X_train.columns,
    fill_value=0
)

X_train = sm.add_constant(X_train)
X_test = sm.add_constant(X_test)

y_train = np.log(train_pdf["sellingprice"])

X_train = X_train.astype(float)
X_test = X_test.astype(float)

```



```
y_train = y_train.astype(float)
```

Sampling Strategy

A stratified sampling approach was used to preserve state representation while reducing dataset size to 5000 for modeling performance.

Training and testing partitions were generated after sampling to maintain representative geographic distributions. A standard random sample was used to split off 20% of the data for testing.

Explore

Exploratory Data Analysis

Initial visualizations focused on the relationships between selling price and major numerical predictors.

```
# Setup a 1x3 grid for the 3 plots
fig, axes = plt.subplots(1, 3, figsize=(18, 6))
fig.suptitle('Automobile Sales Trend Analysis', fontsize=16)

# Plot 1: Most common makes being sold (Top 15)
top_makes = df['make'].value_counts().sort('count', descending=True).head(15).to_pandas()
sns.barplot(data=top_makes, y='make', x='count', hue='make', legend=False,
ax=axes[0], palette='magma')
axes[0].set_title('Top 15 Most Common Makes Sold')
axes[0].set_xlabel('Number of Vehicles Sold')
axes[0].set_ylabel('Make')

# Plot 2: Most common years of vehicle sold
common_years = df['year'].value_counts().sort('count',
descending=True).head(20).sort('year').to_pandas()
sns.barplot(data=common_years, x='year', y='count', hue='year', legend=False,
ax=axes[1], palette='crest')
axes[1].set_title('Most Common Vehicle Years Sold (Top 20)')
axes[1].set_xlabel('Vehicle Year')
axes[1].set_ylabel('Number of Vehicles Sold')
axes[1].tick_params(axis='x', rotation=45)

# Plot 3: Comparison of vehicle age to selling price
age_price = df.groupby('vehicle_age').agg(
    pl.col('sellingprice').mean().alias('mean_price')
).sort('vehicle_age')
# Filter to a reasonable age range to avoid outliers skewing the plot
```

```

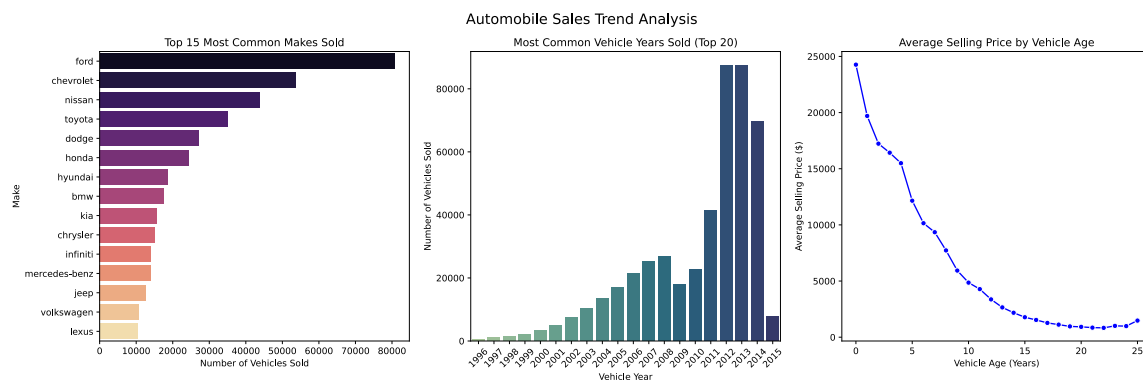
age_price = age_price.filter(pl.col('vehicle_age') <= 30).to_pandas()

sns.lineplot(data=age_price, x='vehicle_age', y='mean_price', ax=axes[2],
marker='o', color='b')
axes[2].set_title('Average Selling Price by Vehicle Age')
axes[2].set_xlabel('Vehicle Age (Years)')
axes[2].set_ylabel('Average Selling Price ($)')

plt.tight_layout()

train_pdf["log_price"] = np.log(train_pdf["sellingprice"])

```



- A log transformation was applied to the selling price.

Vehicle Age vs Selling Price

Vehicle age showed a strong negative relationship with price. Older vehicles generally sold for substantially less than newer vehicles.

```

plt.figure(figsize=(10, 6))

sns.regplot(
    data=train_pdf,
    x="vehicle_age",
    y="log_price",
    scatter_kws={"alpha": 0.15},
    line_kws={"linewidth": 3}
)

plt.title("Vehicle Age vs Selling Price")
plt.xlabel("Vehicle Age (Years)")
plt.ylabel("Selling Price (log $)")
plt.tight_layout()

```



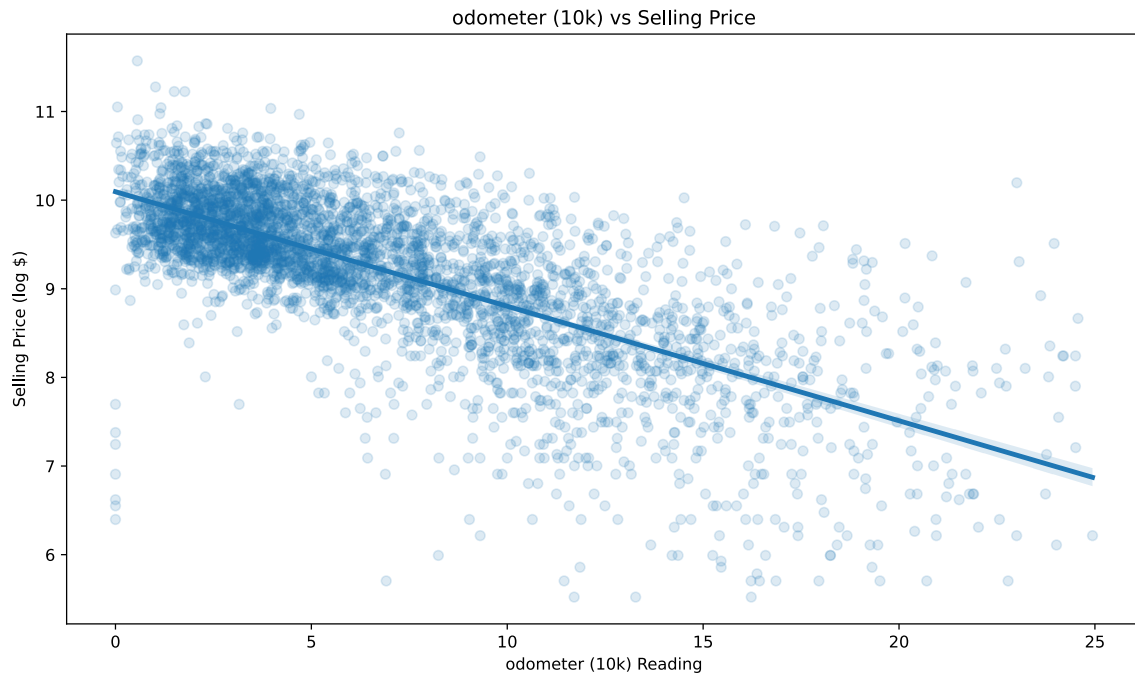
Odometer vs Selling Price

Vehicles with higher mileage generally sold for lower prices. The relationship was weaker than vehicle age but remained statistically significant.

```
plt.figure(figsize=(10, 6))

sns.regplot(
    data=train_pdf,
    x="odometer_10k",
    y="log_price",
    scatter_kws={"alpha": 0.15},
    line_kws={"linewidth": 3}
)

plt.title("odometer (10k) vs Selling Price")
plt.xlabel("odometer (10k) Reading")
plt.ylabel("Selling Price (log $)")
plt.tight_layout()
```



Key Findings

- Vehicle age exhibits the strongest continuous predictor effect.
- Mileage has a meaningful negative effect on vehicle value.
- Vehicle model accounts for substantial differences in selling price.
- Trucks and heavy-duty pickups tend to command premium prices.
- Geographic location contributes measurable price variation.

Reconcile

Assumptions

Multiple linear regression requires:

- Linear relationships
- Independent observations
- Constant variance
- Approximately normal residuals
- Limited multicollinearity

Model Refinements

Several iterations of the model were evaluated.

Adjustments included:

- Consolidating sparse model categories.
- Grouping low-frequency states.
- Transforming odometer readings to improve interpretability.

- Evaluating Variance Inflation Factors (VIFs).

These modifications reduced model complexity while preserving predictive performance.

```
fit_log = sm.OLS(y_train, X_train).fit()

y_test = np.log(test_pdf["sellingprice"])

y_test = y_test.astype(float)

predictions = fit_log.predict(X_test)

residuals = y_test - predictions

vif_data = pd.DataFrame({
    "Variable": X_train.columns,
    "VIF": [variance_inflation_factor(X_train.values, i)
           for i in range(X_train.shape[1])]
})

vif_data.sort_values("VIF", ascending=False)
```

	Variable	VIF
0	const	178.803946
102	state_grouped_fl	6.907212
100	state_grouped_ca	6.474079
63	model_chevrolet_Other	6.085027
9	model_Altima	4.928018
...	...	...
67	model_fiat_Other	1.193466
90	model_saab_Other	1.107573
93	model_smart_Other	1.103110
74	model_isuzu_Other	1.082286
65	model_daewoo_Other	1.040533

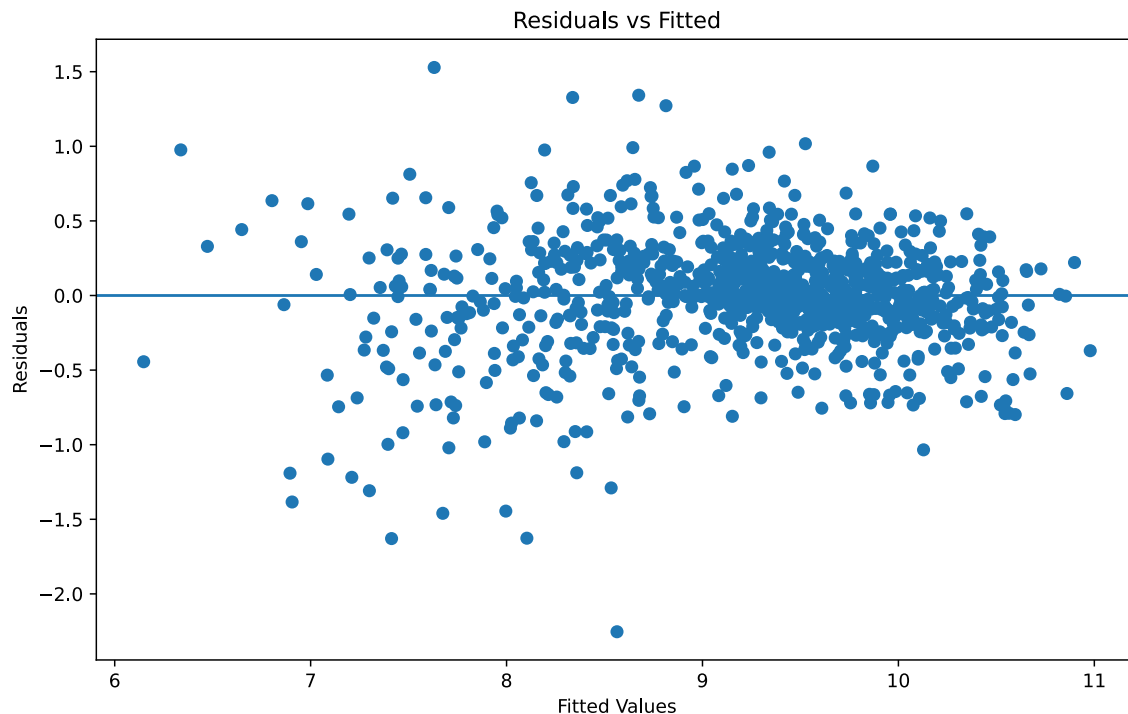
Our original VIF showed high values with mmr, make, and body. Mmr is a market estimate for the worth of the vehicle so it made sense that predictor overpowered all of the other predictors. Likewise, it's a pretty safe assumption that the vehicle model was already capturing the make and body. For example a vehicle model of 'Ram 1500' only could refer to a dodge truck so the model

can easily deduce that data from the vehicle model. Therefore including those in the model was redundant.

Our current model shows no VIF values above or close to 10 so there appear to be no concerns there.

```
plt.figure(figsize=(10, 6))
plt.scatter(predictions, residuals)
plt.axhline(0)
plt.xlabel("Fitted Values")
plt.ylabel("Residuals")
plt.title("Residuals vs Fitted")
```

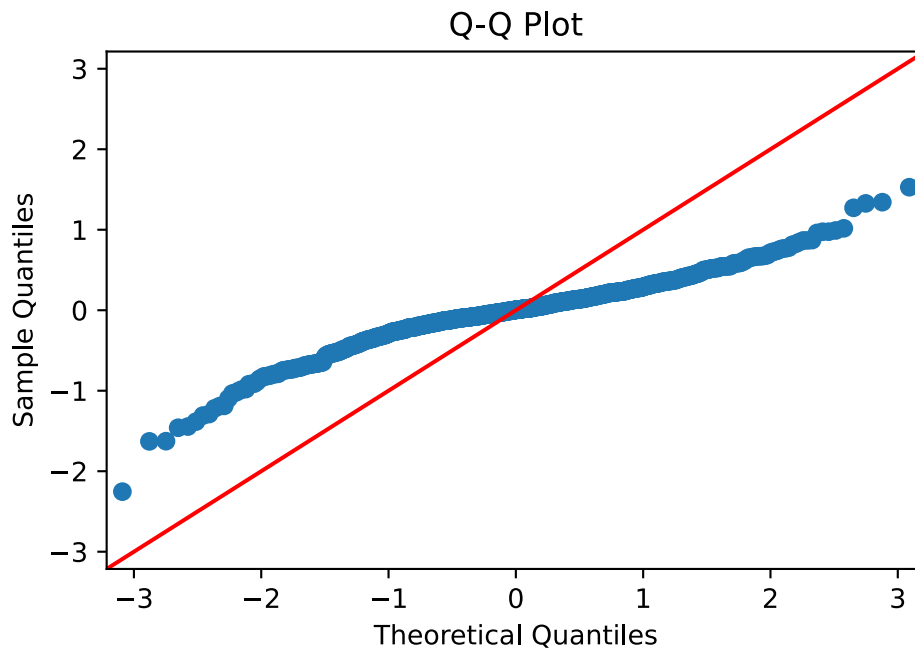
```
Text(0.5, 1.0, 'Residuals vs Fitted')
```



Residuals remained approximately centered around zero with some evidence of heavy tails among high-value vehicles.

```
sm.qqplot(residuals, line='45')
plt.title("Q-Q Plot")
```

```
Text(0.5, 1.0, 'Q-Q Plot')
```



The Q-Q plot indicates that the residuals are not perfectly normally distributed. Rather than following the reference line, the residuals exhibit a noticeable S-shaped pattern, particularly in the tails. This suggests departures from the normality assumption and indicates that the distribution of errors is more concentrated around the mean than would be expected under a normal distribution.

Some departure from normality is expected in used vehicle sales data due to heterogeneous vehicle types, regional market differences, and the presence of categorical predictors such as vehicle model and state. While the log transformation of selling price improved the residual distribution substantially, the normality assumption is not fully satisfied.

Despite these deviations, the large sample size ($n = 4,000$) and the primary objective of prediction rather than inference make the model reasonably robust to moderate violations of residual normality. The residuals remain centered around zero, and other diagnostics did not indicate severe model deficiencies.

Fit

Final Model

The final model included:

- Vehicle age
- Odometer (10,000-mile units)
- Condition score
- Vehicle model
- State group

```
fit_log.summary()
```

Dep. Variable:	sellingprice	R-squared:	0.818
Model:	OLS	Adj. R-squared:	0.812
Method:	Least Squares	F-statistic:	141.6
Date:	Sat, 20 Jun 2026	Prob (F-statistic):	0.00
Time:	22:30:49	Log-Likelihood:	-1715.4
No. Observations:	4000	AIC:	3679.
Df Residuals:	3876	BIC:	4459.
Df Model:	123		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
const	10.0407	0.080	125.821	0.000	9.884	10.197
vehicle_age	-0.1279	0.003	-44.877	0.000	-0.134	-0.122
odometer_10k	-0.0508	0.002	-24.010	0.000	-0.055	-0.047
condition	0.0073	0.001	14.577	0.000	0.006	0.008
model_200	-0.6800	0.097	-6.995	0.000	-0.871	-0.489
model_3 Series	0.0911	0.082	1.113	0.266	-0.069	0.252
model_300	-0.0872	0.100	-0.876	0.381	-0.282	0.108
model_5 Series	0.3066	0.099	3.104	0.002	0.113	0.500
model_Accord	-0.2591	0.082	-3.175	0.002	-0.419	-0.099
model_Altima	-0.4687	0.075	-6.227	0.000	-0.616	-0.321
model_Avenger	-0.7208	0.091	-7.921	0.000	-0.899	-0.542
model_C-Class	0.0999	0.090	1.116	0.264	-0.076	0.275
model_CR-V	0.1006	0.107	0.941	0.347	-0.109	0.310
model_Camry	-0.3019	0.078	-3.865	0.000	-0.455	-0.149
model_Charger	-0.0663	0.098	-0.673	0.501	-0.259	0.127
model_Civic	-0.4153	0.086	-4.853	0.000	-0.583	-0.248
model_Corolla	-0.4822	0.084	-5.756	0.000	-0.646	-0.318
model_Cruze	-0.6591	0.090	-7.338	0.000	-0.835	-0.483
model_E-Class	0.3532	0.096	3.673	0.000	0.165	0.542

model_Edge	-0.0024	0.090	-0.027	0.978	-0.179	0.174
model_Elantra	-0.6615	0.086	-7.688	0.000	-0.830	-0.493
model_Equinox	-0.2747	0.087	-3.153	0.002	-0.446	-0.104
model_Escape	-0.2795	0.077	-3.647	0.000	-0.430	-0.129
model_Expedition	0.1691	0.109	1.558	0.119	-0.044	0.382
model_Explorer	0.0166	0.085	0.195	0.845	-0.150	0.184
model_F-150	0.2495	0.078	3.181	0.001	0.096	0.403
model_F-250 Super Duty	0.5993	0.100	5.968	0.000	0.402	0.796
model_Focus	-0.7664	0.078	-9.873	0.000	-0.919	-0.614
model_Fusion	-0.4933	0.077	-6.412	0.000	-0.644	-0.342
model_G Sedan	-0.0277	0.084	-0.331	0.740	-0.192	0.136
model_Grand Caravan	-0.3861	0.086	-4.490	0.000	-0.555	-0.218
model_Grand Cherokee	0.0804	0.097	0.832	0.406	-0.109	0.270
model_Impala	-0.5320	0.084	-6.326	0.000	-0.697	-0.367
model_Jetta	-0.5260	0.100	-5.246	0.000	-0.723	-0.329
model_Journey	-0.3347	0.095	-3.531	0.000	-0.521	-0.149
model_Liberty	-0.1972	0.109	-1.817	0.069	-0.410	0.016
model_Malibu	-0.5346	0.085	-6.321	0.000	-0.700	-0.369
model_Maxima	-0.2231	0.087	-2.569	0.010	-0.393	-0.053
model_Mazda3	-0.5291	0.099	-5.332	0.000	-0.724	-0.335
model_Mustang	-0.2246	0.103	-2.187	0.029	-0.426	-0.023
model_Odyssey	-0.0549	0.098	-0.562	0.574	-0.247	0.137
model_Optima	-0.4698	0.094	-4.988	0.000	-0.654	-0.285
model_Passat	-0.6219	0.094	-6.632	0.000	-0.806	-0.438
model_RAV4	-0.0871	0.104	-0.836	0.403	-0.291	0.117
model_Rogue	-0.3026	0.096	-3.151	0.002	-0.491	-0.114
model_Sentra	-0.6501	0.090	-7.223	0.000	-0.827	-0.474
model_Sienna	0.1062	0.107	0.991	0.322	-0.104	0.316
model_Silverado 1500	0.2541	0.087	2.932	0.003	0.084	0.424
model_Sonata	-0.5277	0.088	-6.028	0.000	-0.699	-0.356
model_Sorento	-0.3940	0.103	-3.843	0.000	-0.595	-0.193

model_Soul	-0.6376	0.105	-6.050	0.000	-0.844	-0.431
model_Suburban	0.4482	0.104	4.309	0.000	0.244	0.652
model_Tahoe	0.5444	0.100	5.462	0.000	0.349	0.740
model_Taurus	-0.6687	0.105	-6.360	0.000	-0.875	-0.463
model_Town and Country	-0.3667	0.093	-3.946	0.000	-0.549	-0.184
model_Versa	-0.7535	0.108	-6.969	0.000	-0.965	-0.542
model_Wrangler	0.4013	0.092	4.343	0.000	0.220	0.583
model_X5	0.4439	0.108	4.094	0.000	0.231	0.657
model_acura_Other	0.1363	0.089	1.535	0.125	-0.038	0.310
model_audi_Other	0.2036	0.088	2.318	0.021	0.031	0.376
model_bmw_Other	0.3983	0.087	4.569	0.000	0.227	0.569
model_buick_Other	-0.3924	0.093	-4.213	0.000	-0.575	-0.210
model_cadillac_Other	-0.0152	0.081	-0.186	0.852	-0.175	0.145
model_chevrolet_Other	-0.2908	0.073	-3.962	0.000	-0.435	-0.147
model_chrysler_Other	-1.1530	0.099	-11.659	0.000	-1.347	-0.959
model_daewoo_Other	-2.2839	0.385	-5.931	0.000	-3.039	-1.529
model_dodge_Other	-0.1590	0.080	-1.997	0.046	-0.315	-0.003
model_fiat_Other	-0.9297	0.168	-5.519	0.000	-1.260	-0.599
model_ford_Other	-0.2788	0.078	-3.562	0.000	-0.432	-0.125
model_gmc_Other	0.2408	0.083	2.918	0.004	0.079	0.403
model_honda_Other	-0.0278	0.097	-0.287	0.774	-0.217	0.162
model_hummer_Other	0.8827	0.144	6.140	0.000	0.601	1.164
model_hyundai_Other	-0.4713	0.084	-5.607	0.000	-0.636	-0.306
model_infiniti_Other	0.2024	0.084	2.407	0.016	0.038	0.367
model_isuzu_Other	-0.9221	0.278	-3.320	0.001	-1.467	-0.378
model_jaguar_Other	0.2494	0.129	1.935	0.053	-0.003	0.502
model_jeep_Other	-0.3907	0.090	-4.343	0.000	-0.567	-0.214
model_kia_Other	-0.7087	0.083	-8.545	0.000	-0.871	-0.546
model_land rover_Other	0.6535	0.129	5.075	0.000	0.401	0.906
model_lexus_Other	0.3985	0.080	4.990	0.000	0.242	0.555
model_lincoln_Other	-0.1519	0.093	-1.631	0.103	-0.334	0.031

model_mazda_Other	-0.5526	0.089	-6.230	0.000	-0.727	-0.379
model_mercedes-benz_Other	0.5828	0.088	6.641	0.000	0.411	0.755
model_mercury_Other	-0.4185	0.138	-3.042	0.002	-0.688	-0.149
model_mini_Other	-0.3755	0.107	-3.515	0.000	-0.585	-0.166
model_mitsubishi_Other	-0.6594	0.094	-6.988	0.000	-0.844	-0.474
model_nissan_Other	-0.1042	0.078	-1.333	0.183	-0.258	0.049
model_pontiac_Other	-0.7952	0.097	-8.183	0.000	-0.986	-0.605
model_porsche_Other	1.0412	0.150	6.931	0.000	0.747	1.336
model_ram_Other	0.3523	0.158	2.224	0.026	0.042	0.663
model_saab_Other	-0.7784	0.229	-3.393	0.001	-1.228	-0.329
model_saturn_Other	-0.7523	0.101	-7.435	0.000	-0.951	-0.554
model_scion_Other	-0.5546	0.150	-3.701	0.000	-0.848	-0.261
model_smart_Other	-1.1332	0.229	-4.949	0.000	-1.582	-0.684
model_subaru_Other	-0.1688	0.096	-1.750	0.080	-0.358	0.020
model_suzuki_Other	-0.6913	0.138	-5.004	0.000	-0.962	-0.420
model_toyota_Other	0.1528	0.077	1.994	0.046	0.003	0.303
model_volkswagen_Other	-0.2980	0.093	-3.221	0.001	-0.479	-0.117
model_volvo_Other	-0.2249	0.098	-2.284	0.022	-0.418	-0.032
state_grouped_az	0.0218	0.062	0.349	0.727	-0.100	0.144
state_grouped_ca	0.1581	0.044	3.599	0.000	0.072	0.244
state_grouped_co	0.0942	0.067	1.399	0.162	-0.038	0.226
state_grouped_fl	0.1232	0.043	2.842	0.005	0.038	0.208
state_grouped_ga	0.1410	0.047	3.004	0.003	0.049	0.233
state_grouped_il	0.1443	0.050	2.909	0.004	0.047	0.242
state_grouped_in	0.0447	0.078	0.574	0.566	-0.108	0.197
state_grouped_ma	-0.0426	0.066	-0.648	0.517	-0.171	0.086
state_grouped_md	-0.0296	0.059	-0.503	0.615	-0.145	0.086
state_grouped_mi	0.1618	0.054	3.024	0.003	0.057	0.267
state_grouped_mn	0.2002	0.060	3.332	0.001	0.082	0.318
state_grouped_mo	0.1418	0.053	2.678	0.007	0.038	0.246
state_grouped_nc	0.1157	0.051	2.268	0.023	0.016	0.216

state_grouped_ne	0.1334	0.080	1.676	0.094	-0.023	0.290
state_grouped_nj	0.0773	0.049	1.565	0.118	-0.020	0.174
state_grouped_nv	0.1024	0.056	1.836	0.066	-0.007	0.212
state_grouped_ny	0.0479	0.072	0.661	0.508	-0.094	0.190
state_grouped_oh	0.0613	0.050	1.225	0.221	-0.037	0.159
state_grouped_pa	0.1496	0.049	3.054	0.002	0.054	0.246
state_grouped_sc	0.1564	0.076	2.059	0.040	0.007	0.305
state_grouped_tn	0.1271	0.050	2.528	0.011	0.029	0.226
state_grouped_tx	0.0745	0.045	1.646	0.100	-0.014	0.163
state_grouped_va	0.0361	0.059	0.617	0.537	-0.079	0.151
state_grouped_wa	0.0933	0.064	1.462	0.144	-0.032	0.218
state_grouped_wi	0.1179	0.059	1.989	0.047	0.002	0.234

Omnibus:	499.223	Durbin-Watson:	2.020
Prob(Omnibus):	0.000	Jarque-Bera (JB):	3013.924
Skew:	-0.431	Prob(JB):	0.00
Kurtosis:	7.164	Cond. No.	3.85e+03

The fitted model produced:

- $R^2 = 0.818$
- Adjusted $R^2 = 0.812$
- F-statistic = 141.6
- $p < 0.001$

indicating strong overall explanatory power.

Important Continuous Predictors

Variable	Coefficient
Vehicle Age	-0.1279
Odometer (10k miles)	-0.0508
Condition	0.0073

Because the dependent variable is log-transformed, coefficients can be approximately interpreted as percentage changes.

Vehicle Age

Each additional year reduces expected selling price by approximately:

$$100(e^{-0.1279} - 1) \approx -12.0\%$$

Odometer

Each additional 10,000 miles reduces expected selling price by approximately:

$$100(e^{-0.0508} - 1) \approx -5.0\%$$

Condition

Each one-point increase in condition increases expected selling price by approximately:

$$100(e^{0.0073} - 1) \approx 0.7\%$$

Evaluate

Residual Analysis

Residual diagnostics indicated:

- No severe violations of linearity.
- Acceptable residual symmetry.
- Some heavy tails remain.
- No extreme multicollinearity concerns.

The largest VIF values were associated with popular truck categories but remained below commonly cited thresholds requiring corrective action.

Limitations

Several limitations should be noted:

- No MSRP information was available.
- Trim level information was not incorporated.
- Accident history was unavailable.
- Maintenance records were unavailable.
- Auction data may not represent all vehicle markets.

These limitations likely explain a portion of the unexplained variance.

Predict

```
examples = [
    {"make": "Ram", "model": "1500", "vehicle_age": 7.0, "odometer": 115000.0,
     "condition": 30.0, "state": "ut"},
    {"make": "Toyota", "model": "Camry", "vehicle_age": 3.0, "odometer": 42000.0,
     "condition": 35.0, "state": "tx"},
    {"make": "Chevrolet", "model": "2500", "vehicle_age": 2.0, "odometer":
     18000.0, "condition": 45.0, "state": "co"},
    {"make": "Honda", "model": "Civic", "vehicle_age": 10.0, "odometer": 120000.0,
     "condition": 25.0, "state": "ny"}]
```

```

]

def predict_vehicle_price(make: str, model: str, vehicle_age: float, odometer:
float, condition: float, state: str, output_dir: str = "./output"):

    # 1. Load model and metadata
    output_dir = "./output"
    model_path = os.path.join(output_dir, 'car_price_model.pkl')
    metadata_path = os.path.join(output_dir, 'model_metadata.pkl')

    if not os.path.exists(model_path) or not os.path.exists(metadata_path):
        raise FileNotFoundError(
            f"Model or metadata not found in {output_dir}. Please run
trendAnalysis.py first to train and save the model."
        )

    fit_log = sm.load(model_path)
    with open(metadata_path, 'rb') as f:
        metadata = pickle.load(f)

    common_models = metadata['common_models']
    train_columns = metadata['train_columns']
    residual_variance = metadata['residual_variance']

    # 2. Preprocess input parameters
    clean_make = str(make).strip().lower()
    clean_make = make_map.get(clean_make, clean_make)

    # Perform case-insensitive lookup against common_models
    model_lookup = { (m_make, m_model.lower()): m_model for m_make, m_model
in common_models }
    clean_model_lower = str(model).strip().lower()

    # Map to canonical model name if it's common, otherwise collapse to make_Other
    if (clean_make, clean_model_lower) in model_lookup:
        collapsed_model = model_lookup[(clean_make, clean_model_lower)]
    else:
        collapsed_model = f"{clean_make}_Other"

    # Scale odometer to 10k miles units
    odometer_10k = odometer / 10000.0

    # Canonicalize state (lowercase, strip)

```

```

clean_state = str(state).strip().lower()

# 3. Construct the single-row DataFrame for encoding
input_df = pd.DataFrame([
    'vehicle_age': float(vehicle_age),
    'odometer_10k': float(odometer_10k),
    'condition': float(condition),
    'model': collapsed_model,
    'state': clean_state
])

# One-hot encode the categorical columns
input_encoded = pd.get_dummies(input_df, columns=['model', 'state'])

# 4. Align with training columns (fill missing columns with 0, drop extra
columns)
X_new = input_encoded.reindex(columns=train_columns, fill_value=0)

# Set the statsmodels constant term to 1.0
if 'const' in X_new.columns:
    X_new['const'] = 1.0

# Ensure columns are floats
X_new = X_new.astype(float)

# 5. Predict log selling price
predicted_log_price = fit_log.predict(X_new)[0]

# 6. Revert log transformation
# Naive exponentiation yields the median prediction (since E[log(Y)] matches
the median for log-normal)
predicted_median_price = np.exp(predicted_log_price)

# Correcting for log-normal expectation bias yields the mean (expected
value) prediction:
#  $E[Y] = \exp(\mu + \sigma^2 / 2)$ 
predicted_mean_price = np.exp(predicted_log_price + (residual_variance /
2.0))

return {
    "raw_inputs": {
        "make": make,
        "model": model,

```

```

        "vehicle_age": vehicle_age,
        "odometer": odometer,
        "condition": condition,
        "state": state
    },
    "processed_inputs": {
        "canonical_make": clean_make,
        "collapsed_model": collapsed_model,
        "odometer_10k": odometer_10k,
        "clean_state": clean_state
    },
    "predicted_log_price": predicted_log_price,
    "predicted_median_price": predicted_median_price,
    "predicted_expected_price": predicted_mean_price
}

results = []
for ex in examples:
    pred = predict_vehicle_price(
        make=ex["make"],
        model=ex["model"],
        vehicle_age=ex["vehicle_age"],
        odometer=ex["odometer"],
        condition=ex["condition"],
        state=ex["state"],
        output_dir="output" # Points to project-one/output relative to root
    )

    results.append({
        "Vehicle": f"{ex['make']} {ex['model']}",
        "Age (Yrs)": ex["vehicle_age"],
        "Odometer": f"{ex['odometer']:,}",
        "Condition": ex["condition"],
        "State": ex["state"].upper(),
        "Predicted Price": f"${pred['predicted_expected_price']:, .2f}",
        "Trade-In (85-90%)": f"${pred['predicted_expected_price'] * 0.85:, .2f}
- ${pred['predicted_expected_price'] * 0.90:, .2f}"
    })

df_results = pd.DataFrame(results)
from IPython.display import HTML
HTML(df_results.to_html(index=False))

```


Vehicle	Age (Yrs)	Odometer	Condition	State	Predicted Price	Trade-In (85-90%)	
Ram 1500	7.0	115,000.0	30.0	UT	\$6,991.06	\$5,942.40 \$6,291.95	-
Toyota Camry	3.0	42,000.0	35.0	TX	\$12,963.03	\$11,018.57 \$11,666.72	-
Chevrolet 2500	2.0	18,000.0	45.0	CO	\$18,112.87	\$15,395.94 \$16,301.58	-
Honda Civic	10.0	120,000.0	25.0	NY	\$2,954.66	\$2,511.46 \$2,659.20	-

Based off of the given results, we do see that the model does give more value to vehicles in better condition and those that are newer. I'm slightly surprised that it gives more value to certain models over others but it does make sense considering that certain models are more sought after than others. It does indicate that trucks retain more value over sedans. These predictions seem reasonable and can be used to inform purchasing decisions.

Business Implications

The model suggests:

1. Vehicle age is the strongest continuous driver of depreciation.
2. Mileage has a significant but smaller effect.
3. Vehicle model contributes substantial value differences.
4. Trucks generally retain value better than passenger sedans.

Recommendations

Organizations purchasing used vehicles should:

- Prioritize lower-age vehicles when possible.
- Consider mileage alongside age rather than in isolation.
- Use model-specific adjustments when evaluating value.
- Compare observed asking prices against model predictions to identify unusually favorable purchasing opportunities.

Future Work

Future improvements could include:

1. Incorporating original MSRP.
2. Including trim-level information.
3. Adding accident-history variables.
4. Incorporating local economic indicators.

Conclusion

The final regression model explains approximately 82% of the variation in used vehicle selling prices and provides a practical framework for vehicle valuation. Vehicle age, mileage, model, and geographic location all contribute significantly to price determination. The resulting model can support data-driven purchasing and pricing decisions while highlighting areas for future model enhancement.